

Predictra Threat Map Backend

Complete Architectural Analysis & Cognitive Study Guide

This comprehensive guide is designed to dissect and explain every backend file of the Predictra Cyber Threat Map. It employs cognitive load theory, analogies, and active recall strategies to ensure absolute conceptual retention. By the end of this reading, you will not just know the codebase — you will remember its behavior permanently.

Sourced from Predictra Threat Map Codebase

June 2026

Contents

1	The Backend Architecture (The Nervous System)	3
1.1	The Core Pipeline	3
2	Configuration & Database Foundations	5
2.1	Configuration: <code>package.json</code> and <code>.env</code>	5
2.2	Database Connector: <code>config/db.js</code>	5
2.3	Optimized Memory Schema: <code>models/ThreatEvent.js</code>	6
3	The Scrapers (Sensory Receptors)	8
3.1	Scraper Details & Algorithms	8
3.1.1	1. Checkpoint (<code>services/scrapers/checkpoint.js</code>)	8
3.1.2	2. Bitdefender (<code>services/scrapers/bitdefender.js</code>)	9
3.1.3	3. C2 Tracker (<code>services/scrapers/c2tracker.js</code>)	9
3.1.4	4. AlienVault OTX (<code>services/scrapers/alienvault.js</code>)	10
3.1.5	5. Fortinet (<code>services/scrapers/fortinet.js</code>)	10
3.1.6	6. Kaspersky / Feodo Tracker (<code>services/scrapers/kaspersky.js</code>)	10
3.1.7	7. MISP Galaxy (<code>services/scrapers/misp-galaxy.js</code>)	10
3.1.8	8. RansomWatch (<code>services/scrapers/ransomwatch.js</code>)	11
3.1.9	9. URLhaus (<code>services/scrapers/urlhaus.js</code>)	11
4	Sector Enrichment & IP Organization Lookups	12
4.1	Modern WHOIS Lookups: RDAP Protocol with Cache	12
4.2	Sector Classification Logic	13
5	The Orchestrator (<code>server.js</code>)	15
5.1	The Batching Pipeline	15
5.2	The Database Quota Monitor	16

5.3	REST API Endpoints	17
6	Active Recall & Memory Consolidation	18
6.1	Quick-Fire Questions	18
6.2	Mental Exercises	18
6.3	Answer Key	18

Chapter 1

The Backend Architecture (The Nervous System)

To master the Predictra backend, we must first build a high-level mental map. Think of the backend not as isolated files of code, but as a **living organism** — a nervous system that intercepts global threat signals, processes them, and relays them to visual display modules in real-time.

Biological Part	Code Component	Operational Responsibility
Sensory Organs	Scrapers (<code>services/scrapers/*</code>)	Intercepting public cyber threat feeds.
Nerve Center (Brain)	Orchestrator (<code>server.js</code>)	Filtering, queue batching, and routing data.
Long-term Memory	MongoDB Database (<code>models/ThreatEvent</code>)	Saving historic security logs.
Vocal Cords	Server-Sent Events (<code>/api/feed</code>)	Broadcasting threats to frontend maps.

Figure 1.1: Predictra’s Biological Architectural Analogy

1.1 The Core Pipeline

The data flow travels through four distinct stages:

1. **Ingestion:** Scrapers pull raw JSON/Text feeds from security APIs and vendors.
2. **Translation & Jittering:** Raw data is reformatted into a uniform model. Latitude and longitude are given minor *jitter* offsets to avoid visual overlap when plotting.
3. **Enrichment:** The IP is geolocated, its owner organization is looked up via RDAP, and keywords are scanned to classify the victim’s industrial sector (e.g., Healthcare, Finance).
4. **Batching & Broadcast:** Events are buffered and pushed in micro-batches every 300ms to the frontend, while simultaneously written in bulk (batches of 25) to MongoDB.

Memory Anchor: The Mnemonic Acronym: "I-T-E-B"

Remember the four pipeline stages with the word **ITEB**:

- **I** - Ingestion (Scrapers pull data)
- **T** - Translation (Standardizing keys and adding coordinate jitter)
- **E** - Enrichment (RDAP lookups and industry sector mapping)
- **B** - Batching (300ms flushes and bulk MongoDB saves)

Active Recall Challenge: Visualizing the Pipeline

Close your eyes and sketch the path a single threat pulse takes from Fortinet's server to the user's browser screen. Write down what **ITEB** stands for.

Chapter 2

Configuration & Database Foundations

Before the brain can run, its environment and memory structures must be defined. Here we look at how the database is connected, how variables are configured, and why the database storage schema is highly optimized.

2.1 Configuration: `package.json` and `.env`

The `package.json` file defines our external libraries. The key libraries and their purposes are:

- `express` (v5.2.1): The HTTP server framework.
- `mongoose` (v8.23.0): MongoDB Object Modeling, managing schemas and write streams.
- `geoip-lite` (v1.4.10): Fast, local, offline MaxMind-based IP geolocation. We use this instead of querying remote APIs to keep latency near zero.
- `axios` (v1.13.6): Promise-based HTTP client to fetch feeds.
- `socket.io-client` (v2.4.0): For establishing live WebSocket streams with vendors like Bitdefender.
- `eventsource` (v4.1.0): An implementation of the EventSource client API to read Checkpoint's Server-Sent Events (SSE) feed.

2.2 Database Connector: `config/db.js`

This module exposes a single asynchronous function `connectDB()`. Its task is straightforward:

```
1 const connectDB = async () => {
2   try {
3     const uri = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/
      threatmap';
4     await mongoose.connect(uri);
5     console.log('[MongoDB] Connected properly to ${uri}');
6   } catch (error) {
```

```

7     console.error('[MongoDB] Connection error:', error.message);
8     process.exit(1); // Aborts system on database failure
9 }
10 };

```

If the database cannot connect, the process terminates immediately (`process.exit(1)`). This is an intentional defensive programming pattern: running a threat log server without its memory store is a compromised state.

2.3 Optimized Memory Schema: `models/ThreatEvent.js`

This file defines the Mongoose schema for MongoDB. It utilizes **short field names** as database attributes. Let's analyze the schema keys:

Key	Type	Enum / Default	Real Meaning
<code>a_c</code>	Number	Default: 1	Attack Count (occurrences in batch)
<code>a_n</code>	String	Required	Attack Name (signature / CVE details)
<code>a_t</code>	String	'exploit', 'malware', 'phishing'	Attack Type categorization
<code>s_co</code>	String	Required	Source Country Code (2-letter ISO)
<code>s_la</code>	Number	Required	Source Latitude coordinate
<code>s_lo</code>	Number	Required	Source Longitude coordinate
<code>d_co</code>	String	Required	Destination Country Code (2-letter ISO)
<code>d_la</code>	Number	Required	Destination Latitude coordinate
<code>d_lo</code>	Number	Required	Destination Longitude coordinate
<code>s_ip</code>	String	Default: 'unknown'	Attacker Source IP
<code>d_ip</code>	String	Default: 'unknown'	Target Destination IP / Victim Name
<code>meta</code>	Mixed	Default: {}	Arbitrary key-value metadata container
<code>timestamp</code>	Date	Default: <code>Date.now</code>	Event time (Auto-expires in 30 days)
<code>source_api</code>	String	Required	Scraper identifier (e.g., 'fortinet')

Deep Dive: Why use abbreviated keys like `a_c` and `s_la`?

In high-frequency real-time logging, MongoDB stores millions of documents. Because MongoDB stores field names in every single document, using `source_latitude` instead of `s_la` across 10,000,000 records wastes gigabytes of disk space and memory buffers. Furthermore, shorter keys result in a **smaller network payload** size, letting WebSockets and SSE channels stream data to the browser with reduced packet sizes, preventing client-side network choking.

Note the TTL index on the timestamp field:

```

1 timestamp: { type: Date, default: Date.now, expires: 2592000 }

```

The `expires` value `2592000` represents 30 days in seconds ($30 \times 24 \times 3600$). MongoDB automatically purges threat logs older than 30 days. This keeps our database footprint self-cleaning and bounded.

Active Recall Challenge: The Schema Translation

Match the following local abbreviations to their conceptual meaning:

1. `a_n` → _____
2. `a_t` → _____
3. `s_co` → _____
4. `d_lo` → _____

Chapter 3

The Scrapers (Sensory Receptors)

The scrapers are the sensory organs of Predictra. They run concurrently, polling web feeds, connecting to socket gateways, or parsing public intelligence feeds. Let's compare their operations:

Scraper	Source Type	Polling Rate	Default Type	Data Target
fortinet	JSON API Poll	Every 4 sec	exploit	Fortiguard live outbreaks
kaspersky	JSON File Poll	Every 60 sec	malware/exploit	abuse.ch Feodo Tracker
urlhaus	JSON API Poll	Every 90 sec	malware	abuse.ch URLhaus online URLs
alienvault	JSON API Poll	Every 5 min	exploit	OTX Pulse Activity feeds
ransomwatch	JSON API Poll	Every 10 min	malware	Ransomlook.io recent leaks
c2tracker	Plaintext Poll	Every 15 min	malware	montysecurity C2 lists
misp-galaxy	Static JSON Poll	Every 6 hours	random/mixed	MISP galaxy clusters Github
checkpoint	SSE Feed (Client)	Live Stream	exploit/malware	Checkpoint live API gateway
bitdefender	Socket.io Client	Live Stream	exploit/malware	Bitdefender live stream map

3.1 Scraper Details & Algorithms

3.1.1 1. Checkpoint (services/scrapers/checkpoint.js)

Checkpoint provides a direct Server-Sent Events stream. The scraper connects to <https://threatmap-api.checkpoint.com> and listens for incoming messages. It translates incoming attributes directly to Predictra's database parameters.

- **Overlapping Coordinates Fix:** If the source coordinates and target coordinates are identical, the scraper offsets the destination latitude and longitude by a random jitter offset within $[-1, 1]$ degrees:

```
1 if (data.s_la === d_la_mapped && data.s_lo === d_lo_mapped) {  
2     d_la_mapped += (Math.random() - 0.5) * 2;
```

```

3     d_lo_mapped += (Math.random() - 0.5) * 2;
4 }

```

This ensures the visualization shows an arc curve rather than a collapsed point on the globe.

- **Russian Federation Fix:** Checkpoint uses RF for Russia. The scraper transforms this to RU to maintain standard ISO consistency:

```

1 s_co: (data.s_co === 'RF' ? 'RU' : data.s_co) || '??'

```

3.1.2 2. Bitdefender (services/scrapers/bitdefender.js)

Bitdefender operates a live WebSocket feed. We connect to their websocket endpoint using `socket.io-client`.

- **Handshake Details:** The scraper sets up transport to use only `['websocket']` and automatically attempts reconnection every 3 seconds.
- **Subscriptions:** Upon connection, it emits a subscription payload for topics: `botnet`, `portscan`, `telnet`, `ssh`, `rdp`, `vnc`, `mysql`, `mssql`, `http`, `iot`, `iot_botnet`, `infections`, `spam`.
- **One-Sided Coordinate Generation:** Often, Bitdefender events only provide an attacker location OR a victim location. If one side is missing, the scraper generates coordinates for the missing end by applying a jitter offset of up to ± 5 degrees around the known point.
- **Reporting Counter packets:** Every 50 attacks, the scraper broadcasts a `'counter'` payload to update session tallies.

3.1.3 3. C2 Tracker (services/scrapers/c2tracker.js)

C2 Tracker scans active botnet command & control servers. It fetches IP address lists from raw GitHub files maintained by MontySecurity.

- **Targets:** Cobalt Strike, Sliver C2, Havoc C2, Remcos RAT, and Metasploit Framework.
- **Algorithm:** For each tracker, it parses plaintext lists of IPs separated by newlines. To prevent flooding the web application with thousands of locations simultaneously, the scraper extracts a random sample of 10 active IPs per cycle.
- **Geolocation:** The IPs are geolocated using `geoip-lite`.
- **Target Jitter:** Since victim IPs are not provided, it maps attacks to target destinations (e.g., US, UK, DE) with a minor coordinate jitter to keep the lines separated on the map.
- **Staggered Emit:** Pushes updates to the UI using a staggered timer up to 10 seconds:

```

1  setTimeout(() => { broadcast('attack', mappedEvent); }, Math.
    random() * 10000);

```

3.1.4 4. AlienVault OTX (services/scrapers/alienvault.js)

This scraper monitors high-confidence threat reports ("Pulses") logged by the AlienVault Open Threat Exchange community.

- **Poll:** It queries the AlienVault API every 5 minutes.
- **Mapping:** It parses the top 20 pulses, maps the author names, extracts associated malware families and tags, and triggers staggered updates to the frontend.

3.1.5 5. Fortinet (services/scrapers/fortinet.js)

Fortinet maps outbreak-specific threats.

- **Handshake:** The scraper first queries the active outbreak API. It skips the generic "All Outbreaks" (ID 0) to find the first real, active security threat campaign (e.g. specific malware spreading).
- **Poll:** Every 4 seconds, it pulls outbreak event lists. It maps coordinates, vulnerability signatures, severity indexes, and outbreak IDs, and triggers broadcasts.

3.1.6 6. Kaspersky / Feodo Tracker (services/scrapers/kaspersky.js)

Since Kaspersky does not offer a free public API for their cyber map, Predictra matches their telemetry data by polling abuse.ch's **Feodo Tracker**.

- **Source:** A JSON blocklist of active C2 botnet IP addresses updated regularly by abuse.ch.
- **Poll:** Runs every 60 seconds, parses up to 40 active nodes, and maps them. It uses a customized user-agent header (PredictraThreatMap/1.0) as requested by abuse.ch to maintain good API citizenship.

3.1.7 7. MISP Galaxy (services/scrapers/misp-galaxy.js)

Instead of active network feeds, this module integrates structural intelligence. It fetches profiles of Advanced Persistent Threat (APT) groups, ransomware families, and hacking tools from the MISP Galaxy repository on GitHub.

- **Cache:** Refreshes its local database cache every 6 hours.
- **Simulation Generator:** Every 20 seconds, it selects a random threat actor and ransomware profile from the cache. It resolves suspected sponsor countries, identifies target country pools, jitters coordinates, and pushes simulated live-intel arcs.

3.1.8 8. RansomWatch (`services/scrapers/ransomwatch.js`)

This scraper monitors ransomware extortion rings. It leverages the Ransomlook.io API, which compiles postings from active Tor dark web leak sites.

- **Rate:** Polls the API every 10 minutes.
- **Anti-Flood Safeguard:** On the first load, it limits processing to the latest 15 leaks. On subsequent ticks, it calculates the difference using a count tracker (`lastRecordCount`) and emits only newly found posts.

3.1.9 9. URLhaus (`services/scrapers/urlhaus.js`)

URLhaus logs active sites distributing malicious files.

- **Poll:** Queries abuse.ch URLhaus API every 90 seconds.
- **Logic:** Filters URLs with status `'online'`, extracts the domain host, resolves location using local GeoIP search, and pushes events.

Memory Anchor: Why Jitter Coordinates?

Jittering offsets coordinates by a tiny random fraction: `lat + (Math.random() - 0.5) * JITTER_FACTOR`. Without jittering, if 20 attacks hit London or Paris, all 20 lines would point to the exact same coordinate. They would render directly on top of each other, making it look like a single attack. Jittering spreads the lines into a natural-looking "swarm," showing the user the true volume of threats visually.

Active Recall Challenge: Scraper Mechanics

1. Why does Bitdefender require a coordinate generation fallback?
2. What is the purpose of `lastRecordCount` in the RansomWatch scraper?
3. Why do we stagger event emissions in the C2 Tracker scraper using `setTimeout`?

Chapter 4

Sector Enrichment & IP Organization Lookups

Raw threat indicators contain data that is difficult for humans to quickly grasp, such as IP addresses, ports, or raw malware tags. The enrichment engine (`services/enrichment.js`) transforms these raw data points into actionable insights by answering two critical questions:

1. Who owns the IP address?
2. Which industrial sector is being targeted?

4.1 Modern WHOIS Lookups: RDAP Protocol with Cache

Historically, systems used WHOIS queries to lookup IP ownership, which is slow and hard to parse. The enrichment service uses the modern **Registration Data Access Protocol (RDAP)**.

- **Gateway API:** It queries the redirection service `https://rdap.org/ip/{ip}`, which redirects to the proper regional registry (ARIN, RIPE, APNIC, LACNIC, or AFRINIC) and returns JSON data.
- **Parsing Engine:** It checks the registry payload for organization names. It scans the structured `entities` array, flattens card data (`vcardArray`), and falls back to check the main body `name` field.
- **In-Memory Cache (Performance Guard):** Performing RDAP requests for every incoming threat would block scrapers and exceed API rate limits. The system uses a simple JS Map named `rdapCache` to cache results:

```
1  const rdapCache = new Map();
2  // ...
3  if (rdapCache.has(ip)) return rdapCache.get(ip);
4  // ... (perform request)
5  rdapCache.set(ip, orgName);
```

4.2 Sector Classification Logic

To assign an attack to an industry sector (e.g., Healthcare, Finance, Utilities), the service processes threat metadata through a **five-tier fallback classification hierarchy**:

Tier	Classifier Method	Logic & Parameters
1	Target Name Keywords	Checks the victim organization name against lists of keywords.
2	Combined Text Scan	Scans malware signatures, attack descriptions, and threat tags.
3	Port Fallbacks	Assigns infrastructure roles based on destination ports (e.g., Port 443 → Web Services).
4	MISP Galaxy Targets	Maps industry designations from threat actor records.
5	Scraper Source Default	Assigns sector defaults based on scraper origin.

Figure 4.1: Predictra's Sector Enrichment Hierarchy

Let's look at the port-based classification helper inside `getEnrichedSector`:

```

1 if (port) {
2   const p = parseInt(port);
3   if ([80, 443, 8080, 8443].includes(p)) return 'Web Services';
4   if ([3306, 5432, 1433, 27017, 6379].includes(p)) return 'Database
    Services';
5   if ([22, 23, 21, 53, 161].includes(p)) return 'IT Infrastructure';
6   if ([445, 139, 3389].includes(p)) return 'Enterprise Network';
7   if ([25, 587, 465, 110, 143].includes(p)) return 'Email /
    Communication';
8 }
    
```

If all checks fail, the system falls back to the scraper source. For example, events from `ransomwatch` are mapped to `'Finance / Business'` (since ransomware is typically financial extortion), while alerts from `c2tracker` default to `'IT Infrastructure'`.

Memory Anchor: The Sector Map Hierarchy

Use the mnemonic **N-T-P-M-S** (Name, Text, Port, MISP, Source):

- **N** - Name (Check victim name keywords)
- **T** - Text (Check combined text keywords)
- **P** - Port (Fallback to port mapping)
- **M** - MISP (Fallback to MISP Galaxy tags)
- **S** - Source (Fallback to scraper source default)

Active Recall Challenge: Sector Enrichment

1. What is the purpose of `rdapCache`? What would happen if it were removed?
2. What sector is assigned to an attack on port 1433? What about port 3389?
3. Write down the five tiers of sector classification (**N-T-P-M-S**).

Chapter 5

The Orchestrator (server.js)

At the center of the Predictra backend is `server.js`. This orchestrator handles incoming threats, manages connected users, saves logs to the database, and exposes REST endpoints.

5.1 The Batching Pipeline

The system receives dozens of threat updates per second. Sending each update immediately over SSE and writing it to MongoDB causes **computational bottlenecks** — leading to high CPU usage on the server, write lockups in MongoDB, and browser rendering lag. To prevent this, `server.js` implements a batching queue:

```
1 const BATCH_INTERVAL_MS = 300;    // Flush buffer every 300ms
2 const MAX_PENDING = 500;           // Drop oldest events if queue
   overflows
3 const BATCH_DB_INSERT = 25;        // Bulk-insert MongoDB when we have 25
   docs
```

Scrapers → pendingEvents[] & pendingDbDocs[] → SSE Clients (every 300ms)
→ MongoDB (when docs ≥ 25)

Figure 5.1: Visual Representation of the Batching Pipeline

The `flushBatch` function runs on a 300ms loop:

```
1 const flushBatch = () => {
2   if (pendingEvents.length === 0) return;
3   const batch = pendingEvents;
4   pendingEvents = [];
5
6   // Broadcast ONE single message with the entire array
7   const payload = JSON.stringify(batch);
8   clients.forEach(client => {
9     try { client.res.write('event: attacks\ndata: ${payload}\n\n'); }
10    catch (e) {}
11  });
```

```

11
12 // Bulk-insert into MongoDB
13 if (isDatabaseEnabled && !dbSizeLimitReached && pendingDbDocs.length
14     >= BATCH_DB_INSERT) {
15     const docs = pendingDbDocs.splice(0, pendingDbDocs.length);
16     ThreatEvent.insertMany(docs, { ordered: false })
17     .catch(err => console.error('[MongoDB] Bulk insert error:', err.
18         message));
19 }
20 };

```

- **ordered: false:** Tells MongoDB to insert documents in parallel. If one insert fails (e.g. duplicate key), it continues writing the remaining items instead of aborting the entire batch.

5.2 The Database Quota Monitor

If the system runs for weeks, logging every threat could fill the database storage and crash the host machine. To prevent this, the server monitors database size:

```

1 const MAX_DB_SIZE_BYTES = 500 * 1024 * 1024; // 500 MB limit
2 setInterval(async () => {
3     if (!mongoose.connection || mongoose.connection.readyState !== 1)
4         return;
5     try {
6         const stats = await mongoose.connection.db.stats();
7         const size = stats.totalSize || (stats.storageSize + stats.indexSize
8             ) || stats.dataSize;
9         if (size >= MAX_DB_SIZE_BYTES) {
10             if (!dbSizeLimitReached) {
11                 console.warn('[MongoDB] Database size (${(size / 1024 / 1024)}.
12                     toFixed(2)} MB) exceeds 500 MB limit! Disabling database
13                     storage. ');
14                 dbSizeLimitReached = true;
15                 isDatabaseEnabled = false; // Automatically stop saving data
16             }
17         }
18     } catch (err) {
19         console.error('[MongoDB] Error checking db size:', err.message);
20     }
21 }, 60000);

```

This background process checks MongoDB metrics every minute. If storage reaches 500MB, it turns off database saves (`isDatabaseEnabled = false`) while letting live SSE visual streaming continue. This is an elegant safety switch.

5.3 REST API Endpoints

The server exposes several endpoints for frontend modules:

- `GET /api/feed`: Establishes the real-time EventSource connection for the UI client.
- `GET /api/history`: Returns up to 200 historical events matching search queries (`q`, `ip`, or `country`) using case-insensitive RegExp scans on database properties.
- `GET /api/stats`: Returns counts categorized by attack type, country of origin, destination target, attack vectors, and scraper source API.
- `GET /api/analytics/sectors`: Classifies and aggregates sector-specific attack breakdowns (limited to 5,000 events to maintain performance).
- `GET /api/galaxy/*`: Serves cached data for Threat Actors, Ransomware families, and Adversary tools.

Active Recall Challenge: Server Management

1. Explain the problem that the 300ms batching loop solves.
2. What is the significance of the `{ ordered: false }` configuration when performing `insertMany`?
3. What happens to the database write pipeline if MongoDB usage exceeds 500MB?

Chapter 6

Active Recall & Memory Consolidation

To finalize our guide, let's complete a comprehensive review. Try to answer these questions from memory before checking the source code.

6.1 Quick-Fire Questions

1. **Question:** Which scraper connects directly to a WebSocket client stream?
2. **Question:** Why does the server check database size every 60 seconds?
3. **Question:** What is the purpose of `rdapCache` inside `enrichment.js`?
4. **Question:** Why does the database schema specify `expires: 2592000`?
5. **Question:** Under what abbreviation is the attack name saved in MongoDB?

6.2 Mental Exercises

- **Exercise 1:** Describe to a peer why coordinate jittering is critical for geographic visualizers.
- **Exercise 2:** Walk through the five steps of the sector enrichment fallback pipeline.
- **Exercise 3:** Contrast the data collection methods of `checkpoint.js` (SSE stream) versus `c2tracker.js` (plaintext lists).

6.3 Answer Key

1. Bitdefender (`bitdefender.js`) uses `socket.io-client`.
2. To prevent storage depletion by turning off MongoDB writes once the database size reaches 500MB.

3. Caches IP organization ownership queries to avoid rate-limiting and minimize lookup latency.
4. It instructs MongoDB to auto-expire and delete document logs after 30 days.
5. It is saved under the key `a_n`.